

## LAB 3

### I. Objectives

After completing this lab, students will be able to:

#### Knowledge

1. Understand the classification problem and the goal of predicting labels from input data
2. Understand evaluation metrics for classification models such as: Accuracy, Precision, Recall, and F1-score
3. Understand the working principles of algorithms:
  - **Decision Tree:** splits data based on criteria such as Gini index or Entropy
  - **Naive Bayes:** a probabilistic model based on Bayes' theorem with the assumption of feature independence

#### Skills

- Build classification models from data
- Train models and make predictions using Decision Tree and Naive Bayes algorithms
- Evaluate and compare model performance using different metrics
- Analyze results and select the most appropriate model for a given problem

### II. Theoretical Summary

Classification is a problem in data mining and machine learning, where the goal is to build a model capable of assigning an object to a specific class based on its features.

In a classification problem, each sample has:

- Input attributes (also called features)
- A class label indicating which category the sample belongs to

From labeled data, we train a model so that when encountering new, unseen data, the model can correctly predict its class.

In essence, classification is the task of learning a mapping function:

$$f(X) = y$$

where:

- $X$  is the set of input features
- $y$  is the output class label

An important point is that the model should not simply memorize the training data, but must generalize well. This means that when encountering new data, the model can still make accurate predictions with a high success rate.

## 1. Types of Classification Problems

### Binary Classification

There are only two classes.

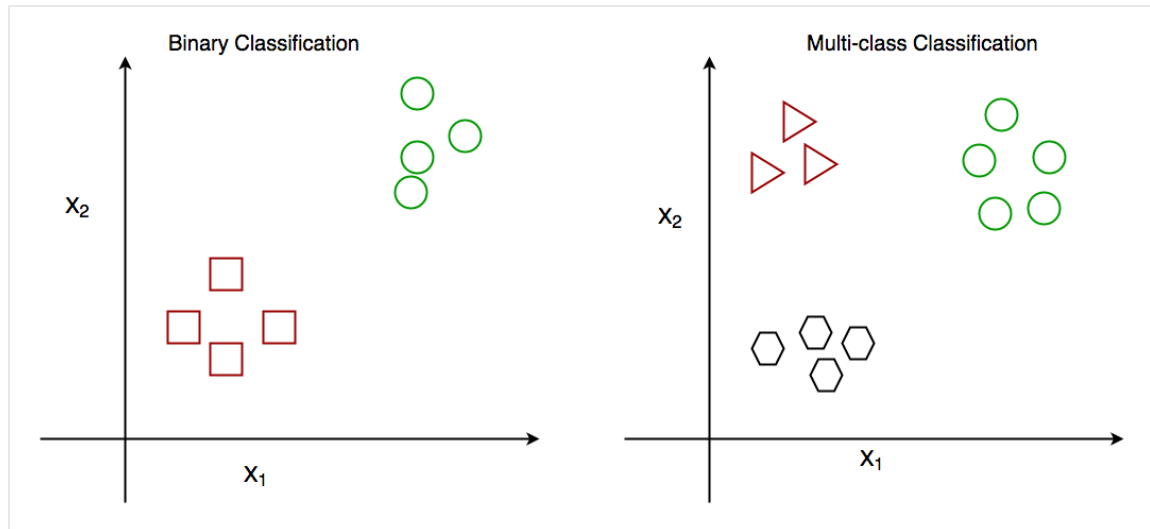


### Examples:

- Disease / No disease
- Pass / Fail
- Spam email / Not spam

## Multi-class Classification

There are more than two classes.



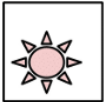
### Examples:

- Flower classification: rose, daisy, orchid
- Handwritten digit classification: 0, 1, 2, ..., 9

## Multi-label Classification

A sample can belong to multiple classes simultaneously.

**Example:** An image may contain a person, a car, and a tree at the same time

| Multi-Class |                                                                                                                                                                                                                                                             | Multi-Label                                                                                                                                                                                                                                                      |  |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| C = 3       | Samples                                                                                                                                                                                                                                                     | Samples                                                                                                                                                                                                                                                          |  |
|             |    |    |  |
|             | Labels (t)<br>[0 0 1] [1 0 0] [0 1 0]                                                                                                                                                                                                                       | Labels (t)<br>[1 0 1] [0 1 0] [1 1 1]                                                                                                                                                                                                                            |  |

## 2. Evaluation of Classification Problems

In classification, after building a model, we need to evaluate how well it performs on data. To do this, the dataset is typically divided into training and testing sets.

### Training Set

The training set is used for the model to learn patterns from input data and output labels. During this phase, the model adjusts its parameters based on labeled samples to distinguish between different classes.

### Test Set

The test set is not used during training. It is only used to evaluate the model after training is complete. The purpose is to check whether the model can make correct predictions on new data. If the model performs well on the training set but poorly on the test set, it may indicate overfitting, meaning the model **memorizes training data but does not generalize well**.

### Importance of Data Splitting

Splitting data into training and test sets helps to:

- Objectively evaluate model performance
- Test prediction ability on unseen data
- Detect overfitting
- Compare the performance of different models

If the test set is not separated, evaluation results may be overly optimistic since the model is tested on data it has already seen.

## 4. Data Splitting Ratios

In practice, common data splitting ratios include:

- 80% training, 20% testing
- 70% training, 30% testing
- 75% training, 25% testing

The chosen ratio depends on dataset size:

- Large datasets → more data can be used for training

- Small datasets → careful splitting is needed to ensure enough data for both sets

## 5. Evaluation on the Test Set

After training the model, we input test samples into the model to predict labels.

Then, we compare predicted labels with true labels to compute evaluation metrics such as:

- Accuracy
- Precision
- Recall
- F1-score

These metrics indicate how well the model performs and whether it is balanced across classes.

### Evaluation Metrics in Classification

Let:

- TP: True Positive (correctly predicted positive)
- TN: True Negative (correctly predicted negative)
- FP: False Positive (incorrectly predicted as positive)
- FN: False Negative (incorrectly predicted as negative)

#### Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

**Meaning:** The proportion of correctly predicted samples over the total dataset.

#### Precision

$$Precision = \frac{TP}{TP + FP}$$

**Meaning:** Among the samples predicted as positive, how many are actually correct.

## Recall

$$Recall = \frac{TP}{TP + FN}$$

**Meaning:** Among the actual positive samples, how many are correctly identified by the model.

## F1-score

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

It can also be written directly using TP, FP, FN:

$$F1 = \frac{2TP}{2TP + FP + FN}$$

**Meaning:** The harmonic mean of Precision and Recall, balancing both metrics.

## 3. Classification using Decision Trees

A decision tree is a classification model with a structure similar to a tree. Each node in the tree represents a test on an attribute, each branch represents the outcome of that test, and each leaf represents the final predicted class.

The process of building a decision tree involves repeatedly splitting the data based on different attributes, with the goal of making the data in each branch easier to predict.

At each node, the algorithm must decide:

**Which attribute should be chosen to split the data?**

This is the most important factor affecting the quality of the decision tree.

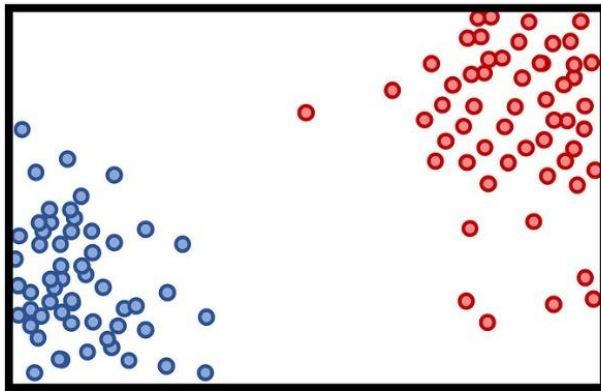
To make this choice, we need a criterion to measure how “good” a split is. These criteria are based on a common idea:

**After splitting, the data should become less uncertain.**

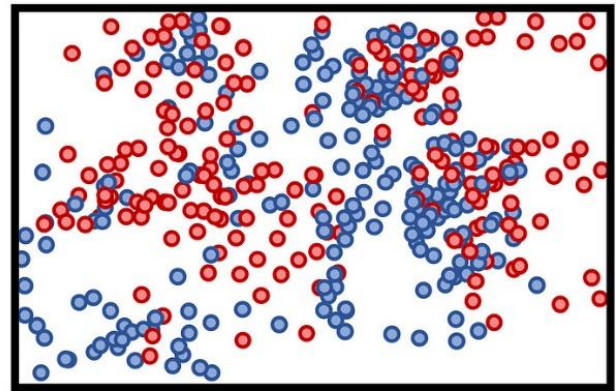
## Entropy

Entropy is a measure of the level of uncertainty in the data.

- If the outcome is easy to predict → Entropy is low
- If the outcome is hard to predict → Entropy is high



Low Entropy



High Entropy

### Entropy Formula

$$H(S) = - \sum_{i=1}^k p_i \log_2 p_i$$

Where:

- $p_i$ : probability of class i
- k: number of classes

### Objective of Building a Decision Tree

The core objective when constructing a decision tree is:

**To split the data in such a way that the uncertainty (Entropy) is reduced as much as possible after each split.**

In other words, the process of building a tree transforms:

- From the initial state: data is mixed and hard to predict (high Entropy)
- To the state after splitting: data in each branch becomes more homogeneous and easier to predict (low Entropy)

**Example:** Predicting a disease (A/B) based on:

- Symptom
- Gender

| ID | Symptom | Gender | Class |
|----|---------|--------|-------|
| 1  | Fever   | Male   | A     |
| 2  | Fever   | Female | A     |
| 3  | Fever   | Male   | A     |
| 4  | Cough   | Male   | B     |
| 5  | Cough   | Female | B     |
| 6  | Cough   | Male   | B     |
| 7  | Fever   | Female | A     |
| 8  | Cough   | Female | B     |

### Initial Distribution

- A: 4
- B: 4

**Ratio:** 50–50 → High Entropy (hard to predict)

**Suppose we choose Gender as the splitting attribute:**

- For **Gender = Male:**

| ID | Symptom | Gender | Class |
|----|---------|--------|-------|
| 1  | Fever   | Male   | A     |
| 3  | Fever   | Male   | A     |
| 4  | Cough   | Male   | B     |
| 6  | Cough   | Male   | B     |

**Ratio:** 50–50 → Entropy remains high

- For **Gender = Female:**

| ID | Symptom | Gender | Class |
|----|---------|--------|-------|
| 2  | Fever   | Female | A     |
| 5  | Cough   | Female | B     |
| 7  | Fever   | Female | A     |
| 8  | Cough   | Female | B     |

**Ratio:** 50–50 → Entropy remains high

Suppose we choose **Symptom** as the splitting attribute:

For **Symptom = Fever**:

| ID | Symptom | Gender | Class |
|----|---------|--------|-------|
| 1  | Fever   | Male   | A     |
| 2  | Fever   | Female | A     |
| 3  | Fever   | Male   | A     |
| 7  | Fever   | Female | A     |

**Ratio:** 100% class A → Entropy = 0

For **Symptom = Cough**:

| ID | Symptom | Gender | Class |
|----|---------|--------|-------|
| 4  | Cough   | Male   | B     |
| 5  | Cough   | Female | B     |
| 6  | Cough   | Male   | B     |
| 8  | Cough   | Female | B     |

**Ratio:** 100% class B → Entropy = 0

**Common Splitting Criteria:**

**Information Gain measures:**

Which attribute reduces entropy the most

$$G(S, A) = E(S) - \sum_{i=1}^m f_s(A_i) E(S_{(A_i)})$$

Where:

- $G(S, A)$ : information gain of dataset  $S$  when splitting by attribute  $A$
- $E(S)$ : entropy of dataset  $S$ , calculated as:

$$E(S) = - \sum_{j=1}^n f_s(A_i) \log_2 f_s(A_i)$$

- $m$ : number of distinct values of attribute  $A$
- $A_i$ : number of samples corresponding to each value  $i$  of attribute  $A$
- $f_S(A_i)$ : proportion of samples with value  $A_i$  in dataset  $S$
- $S_{A_i}$ : subset of  $S$  containing all samples with value  $A_i$

## Gini Index

Some decision tree algorithms such as CART use the Gini index instead of Entropy.

Gini measures:

**The probability of incorrect classification if a sample is randomly labeled according to the distribution**

- High Gini  $\rightarrow$  higher chance of misclassification
- Low Gini  $\rightarrow$  lower error

Gini Formula for Dataset  $S$ :

$$Gini(S) = 1 - \sum_j p(j | S)^2$$

Where  $p(j | S)$  frequency of class  $j$  in dataset  $S$

## Gini for a Split

When splitting node  $A$  into  $k$  branches, the quality of the split is calculated as:

$$Gini_A(S) = \sum_{i=1}^k \frac{n_i}{n} Gini(i)$$

Where:

$n_i$  : number of samples in node  $i$

$n$  : total number of samples in node  $A$

## 4. Classification using Naive Bayes

Naive Bayes is a probabilistic classification model. It uses Bayes' theorem to predict the label of a data sample based on its input features. The main idea is to compute the probability that a sample belongs to each class and then select the class with the highest probability.

Unlike decision trees, classification in Naive Bayes does not rely on splitting the data. Instead, it is based on calculating the probability of each class using the learned data.

For each sample to be predicted, the algorithm must decide:

**Which class has the highest probability for this sample?**

This is the most important factor affecting the classification result of Naive Bayes.

To make this decision, we need to compute the posterior probability of each class based on the input data.

Bayes' Theorem:

$$P(C | x) = \frac{P(x | C) P(C)}{P(x)}$$

Where:

- $P(C | x)$ : probability that sample  $x$  belongs to class  $C$  (posterior probability)
- $P(x | C)$ : probability of observing data  $x$  given class  $C$  (likelihood)
- $P(C)$ : prior probability of class  $C$
- $P(x)$ : probability of the data (normalizing constant)

### III. Illustrative Example

Consider the following dataset for evaluating the risk level of loan applications at a bank:

| Income | Occupation | Marital Status | House Ownership | Risk Level |
|--------|------------|----------------|-----------------|------------|
| Low    | Transport  | Single         | Yes             | High Risk  |
| Low    | Transport  | Single         | No              | High Risk  |
| High   | Transport  | Single         | Yes             | Low Risk   |
| Medium | Media      | Single         | Yes             | Low Risk   |
| Medium | Business   | Married        | Yes             | Low Risk   |
| Medium | Business   | Married        | No              | High Risk  |
| High   | Business   | Married        | No              | Low Risk   |
| Low    | Media      | Single         | Yes             | High Risk  |
| Low    | Business   | Married        | Yes             | Low Risk   |
| Medium | Media      | Married        | Yes             | Low Risk   |

|        |           |         |     |           |
|--------|-----------|---------|-----|-----------|
| Low    | Media     | Married | No  | Low Risk  |
| High   | Media     | Single  | No  | Low Risk  |
| High   | Transport | Married | Yes | Low Risk  |
| Medium | Media     | Single  | No  | High Risk |
| High   | Business  | Single  | Yes | High Risk |

In this dataset, the “**Risk Level**” column is the **target (decision attribute)**.

## 1. Decision Tree

### Building a Decision Tree using Information Gain

Initially, the dataset  $S$  consists of 15 samples, including 9 samples belonging to the **Low Risk** class and 6 samples belonging to the **High Risk** class.

The entropy of dataset  $S$  is calculated as:

$$E(S) = -\frac{9}{15} \log_2 \frac{9}{15} - \frac{6}{15} \log_2 \frac{6}{15} \approx 0.97$$

### Considering the attribute Income

The Income attribute has 3 values: Low, Medium, High.

- Low: 5 samples (3 high, 2 low)
- Medium: 5 samples (2 high, 3 low)
- High: 5 samples (1 high, 4 low)

Entropy:

$$E(S_{low}) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} \approx 0.971$$

$$E(S_{medium}) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \approx 0.971$$

$$E(S_{high}) = -\frac{1}{5} \log_2 \frac{1}{5} - \frac{4}{5} \log_2 \frac{4}{5} \approx 0.722$$

Information Gain:

$$G(S, \text{Income}) = 0.97 - \frac{5}{15} \times 0.971 - \frac{5}{15} \times 0.971 - \frac{5}{15} \times 0.722 \approx 0.083$$

### Considering the attribute Occupation

Values: Transport, Media, Business

- Transport: 4 samples (2 high, 2 low)
- Media: 6 samples (2 high, 4 low)
- Business: 5 samples (2 high, 3 low)

Entropy:

$$E(S_{transport}) = -\frac{2}{4}\log_2 \frac{2}{4} - \frac{2}{4}\log_2 \frac{2}{4} = 1$$

$$E(S_{media}) = -\frac{2}{6}\log_2 \frac{2}{6} - \frac{4}{6}\log_2 \frac{4}{6} \approx 0.918$$

$$E(S_{business}) = -\frac{2}{5}\log_2 \frac{2}{5} - \frac{3}{5}\log_2 \frac{3}{5} \approx 0.971$$

Information Gain:

$$G(S, \text{Occupation}) = 0.97 - \frac{4}{15} \times 1 - \frac{6}{15} \times 0.918 - \frac{5}{15} \times 0.971 \approx 0.013$$

### Considering the attribute Marital Status

Values: Single, Married

- Single: 8 samples (5 high, 3 low)
- Married: 7 samples (1 high, 6 low)

Entropy:

$$E(S_{single}) = -\frac{5}{8}\log_2 \frac{5}{8} - \frac{3}{8}\log_2 \frac{3}{8} \approx 0.954$$

$$E(S_{married}) = -\frac{1}{7}\log_2 \frac{1}{7} - \frac{6}{7}\log_2 \frac{6}{7} \approx 0.6$$

Information Gain:

$$G(S, \text{Marital Status}) = 0.97 - \frac{8}{15} \times 0.954 - \frac{7}{15} \times 0.6 \approx 0.186$$

## Considering the attribute House Ownership

Values: No, Yes

- No: 6 samples (3 high, 3 low)
- Yes: 9 samples (3 high, 6 low)

Entropy:

$$E(S_{no}) = -\frac{3}{6} \log_2 \frac{3}{6} - \frac{3}{6} \log_2 \frac{3}{6} = 1$$
$$E(S_{yes}) = -\frac{3}{9} \log_2 \frac{3}{9} - \frac{6}{9} \log_2 \frac{6}{9} \approx 0.918$$

Information Gain:

$$G(S, \text{House Ownership}) = 0.97 - \frac{6}{15} \times 1 - \frac{9}{15} \times 0.918 \approx 0.02$$

Among the four considered attributes, **Marital Status** has the highest Information Gain. Therefore, this attribute is selected as the root node split of the decision tree.

The dataset is now divided into two subsets corresponding to the two branches of the tree:

- Single: 8 samples
- Married: 7 samples

### For the first branch (Single)

Considering the attribute **Income**, we obtain:

$$E(S_{low}) = 0; E(S_{medium}) = 1; E(S_{high}) \approx 0.92; G(S_{single}, \text{Income}) \approx 0.36$$

For the attributes **Occupation** and **House Ownership**:

$$E(S_{transport}) \approx 0.92; E(S_{media}) = 1; E(S_{business}) = 0$$
$$G(S_{single}, \text{Occupation}) \approx 0.11$$
$$E(S_{no}) \approx 0.92; E(S_{yes}) \approx 0.971; G(S_{single}, \text{House Ownership}) \approx 0.003$$

Thus, for this branch, we select the **Income** attribute for splitting.

- For **Income = Low**, all samples belong to the **High Risk** class → this branch becomes a leaf node and no further splitting is needed.

- The remaining branches corresponding to **Medium** and **High** will continue to be expanded.

The branch corresponding to the value **High** includes 3 data samples. Considering the remaining attributes, we have:

$$E(S_{transport}) = 0; E(S_{media}) = 0; E(S_{business}) = 0;$$

$$G(S_{high}, Occupation) \approx 0.92$$

$$E(S_{no}) = 0; E(S_{yes}) = 1; G(S_{high}, Occupation) \approx 0.252$$

With the higher Information Gain value, the tree continues to split based on the **Occupation** attribute. At this point:

- The branches **Transport** and **Media** lead to leaf nodes with **Low Risk**
- The branch **Business** leads to a leaf node with **High Risk**

The branch corresponding to the value **Medium** includes 2 data samples. Continuing to compute Information Gain:

$$E(S_{media}) = 1; G(S_{medium}, Occupation) = 0$$

$$E(S_{no}) = 0; E(S_{yes}) = 0; G(S_{medium}, House Ownership) = 1$$

This branch is split based on the **House Ownership** attribute:

- The branch **No** leads to a leaf node with **High Risk**
- The branch **Yes** leads to a leaf node with **Low Risk**

Returning to the **Married** branch (from the root), we have 7 data samples. Considering the attributes **Income**, **Occupation**, and **House Ownership**, we obtain:

$$E(S_{low}) = 0; E(S_{medium}) \approx 0.92; E(S_{high}) = 0; G(S_{married}, Income) \approx 0.2$$

$$E(S_{transport}) = 0; E(S_{transport}) = 0; E(S_{business}) \approx 0.811;$$

$$G(S_{married}, Occupation) \approx 0.128$$

$$E(S_{no}) \approx 0.92; E(S_{yes}) = 0; G(S_{married}, House Ownership) \approx 0.2$$

In this branch, two attributes have equal highest Information Gain: **Income** and **House Ownership**. We choose **Income** to continue.

- For **Income = Low** and **Income = High**, the tree leads directly to leaf nodes with **Low Risk**

We only need to further develop the tree for **Income = Medium**:

$$E(S_{media}) = 0; E(S_{business}) = 1; G(S_{medium}, \text{Occupation}) \approx 0.252$$

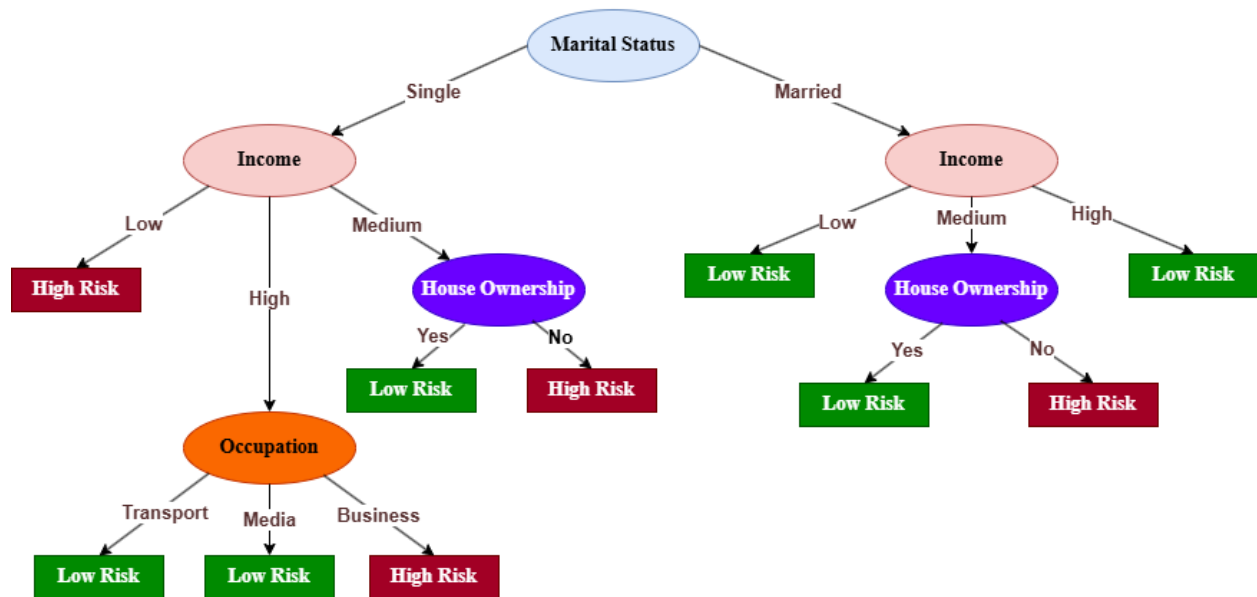
$$E(S_{no}) = 0; E(S_{yes}) = 0; G(S_{medium}, \text{House Ownership}) \approx 0.92$$

We select **House Ownership** as the splitting attribute. At this point:

- For value **No**, the result is always **High Risk**
- For value **Yes**, the result is always **Low Risk**

Thus, the algorithm terminates.

The resulting decision tree is illustrated in the figure below.



### Building a Decision Tree using Gini Index

Based on the statistics from question 2, we compute the Gini index for each attribute to determine the best splitting attribute.

Considering the attribute **Income**, we have:

$$Gini(S_{low}) = 1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2 = 0.48$$

$$Gini(S_{i_{medium}}) = 1 - \left(\frac{2}{5}\right)^2 - \left(\frac{3}{5}\right)^2 = 0.48$$

$$Gini(S_{high}) = 1 - \left(\frac{1}{5}\right)^2 - \left(\frac{4}{5}\right)^2 = 0.32$$

$$Gini_{Income}(S) = \frac{5}{15} \times 0.48 + \frac{5}{15} \times 0.48 + \frac{5}{15} \times 0.32 = 0.427$$

Considering the attribute **Occupation**, we have:

$$Gini(S_{transport}) = 1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2 = 0.5$$

$$Gini(S_{media}) = 1 - \left(\frac{2}{6}\right)^2 - \left(\frac{4}{6}\right)^2 \approx 0.444$$

$$Gini(S_{business}) = 1 - \left(\frac{2}{5}\right)^2 - \left(\frac{3}{5}\right)^2 = 0.48$$

$$Gini_{Occupation}(S) = \frac{4}{15} \times 0.5 + \frac{6}{15} \times 0.444 + \frac{5}{15} \times 0.48 \approx 0.471$$

Considering the attribute **Marital Status**, we have:

$$Gini(S_{single}) = 1 - \left(\frac{5}{8}\right)^2 - \left(\frac{3}{8}\right)^2 \approx 0.469$$

$$Gink(S_{married}) = 1 - \left(\frac{1}{7}\right)^2 - \left(\frac{6}{7}\right)^2 \approx 0.245$$

$$Gini_{Marital\ Status}(S) = \frac{8}{15} \times 0.469 + \frac{7}{15} \times 0.245 \approx 0.403$$

Considering the attribute **House Ownership**, we have:

$$Gini(S_{no}) = 1 - \left(\frac{3}{6}\right)^2 - \left(\frac{3}{6}\right)^2 = 0.5$$

$$Gini(S_{yes}) = 1 - \left(\frac{3}{9}\right)^2 - \left(\frac{6}{9}\right)^2 \approx 0.444$$

$$Gini_{House\ Ownership}(S) = \frac{6}{15} \times 0.5 + \frac{9}{15} \times 0.444 \approx 0.467$$

We select the attribute with the lowest Gini index, which is **Marital Status**, and split the tree accordingly.

Similar to above, for the **Single** branch:

$$Gini_{\text{Income}}(S_{\text{single}}) = \frac{3}{8} \times 0 + \frac{2}{8} \times 0.5 + \frac{3}{8} \times 0.444 \approx 0.292$$

$$Gini_{\text{Occupation}}(S_{\text{single}}) = \frac{3}{8} \times 0.444 + \frac{4}{8} \times 0.5 + \frac{1}{8} \times 0 \approx 0.417$$

$$Gini_{\text{House Ownership}}(S_{\text{single}}) = \frac{3}{8} \times 0.444 + \frac{5}{8} \times 0.48 \approx 0.467$$

We choose the **Income** attribute and continue growing the tree.

For the **Medium** branch:

$$Gini_{\text{Occupation}}(S_{\text{medium}}) = \frac{2}{2} \times 0.5 = 0.5$$

$$Gini_{\text{House Ownership}}(S_{\text{medium}}) = \frac{1}{2} \times 0 + \frac{1}{2} \times 0 = 0$$

In this branch, we choose **House Ownership** as the splitting attribute, and all branches lead to leaf nodes.

**Continuing with the Married branch:**

$$Gini_{\text{Income}}(S_{\text{married}}) = \frac{2}{7} \times 0 + \frac{3}{7} \times 0.444 + \frac{2}{7} \times 0.5 \approx 0.19$$

$$Gini_{\text{Occupation}}(S_{\text{married}}) = \frac{1}{7} \times 0 + \frac{2}{7} \times 0 + \frac{4}{7} \times 0.375 \approx 0.214$$

$$Gini_{\text{House Ownership}}(S_{\text{married}}) = \frac{3}{7} \times 0.444 + \frac{4}{7} \times 0 \approx 0.19$$

The Gini indices of **Income** and **House Ownership** are equal and lowest. We choose **House Ownership** to split.

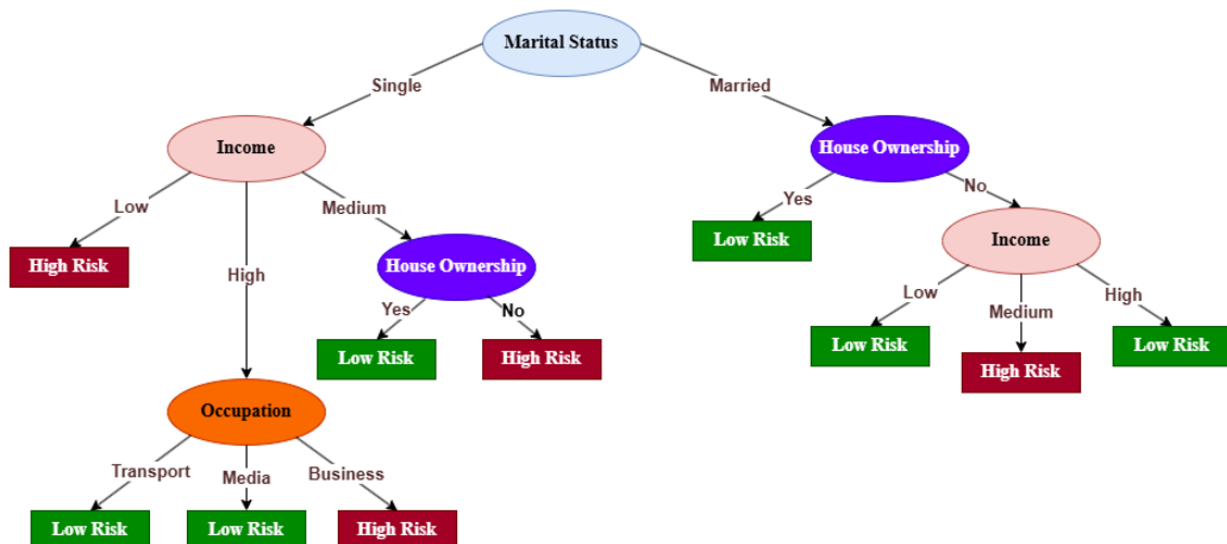
- The branch with value **Yes** leads to a leaf node with **Low Risk**

For the branch **No**, we have:

$$Gini_{\text{Occupation}}(S_{\text{no}}) = \frac{1}{3} \times 0 + \frac{2}{3} \times 0.5 \approx 0.333$$

$$Gini_{\text{Income}}(S_{\text{no}}) = \frac{1}{3} \times 0 + \frac{1}{2} \times 0 = 0$$

We choose **Income** to continue splitting. At this point, all branches lead to leaf nodes. The resulting decision tree is obtained as follows:



## 2. Naïve Bayes Method

Consider each record (profile) one by one. Based on Bayes' theorem, compute the probability of each **Risk Level** and select the class with the highest probability.

### Profile 1:

$X$

= {Income = Medium, Occupation = Transport, Marital Status = Single, House Ownership = Yes}

We need to compute:

$$P(\text{Risk} = \text{High} \mid X) = \frac{P(X \mid \text{Risk} = \text{High}) \cdot P(\text{Risk} = \text{High})}{P(X)}$$

Compare with:

$$P(\text{Risk} = \text{Low} \mid X) = \frac{P(X \mid \text{Risk} = \text{Low}) \cdot P(\text{Risk} = \text{Low})}{P(X)}$$

Since the denominators are the same, we only compare the numerators.

Applying **Laplace smoothing**:

**High Risk:**

$$P(\text{Risk} = \text{High}) = \frac{6 + 1}{15 + 2} = \frac{7}{17} \approx 0.412$$

$$P(\text{Income} = \text{Medium} \mid \text{Risk} = \text{High}) = \frac{2 + 1}{6 + 3} = \frac{1}{3} \approx 0.333$$

$$P(\text{Occupation} = \text{Transport} \mid \text{Risk} = \text{High}) = \frac{2 + 1}{6 + 3} = \frac{1}{3} \approx 0.333$$

$$P(\text{Marital Status} = \text{Single} \mid \text{Risk} = \text{High}) = \frac{5 + 1}{6 + 2} = \frac{3}{4} = 0.75$$

$$P(\text{House Ownership} = \text{Yes} \mid \text{Risk} = \text{High}) = \frac{3 + 1}{6 + 2} = \frac{1}{2} = 0.5$$

$$P(X \mid \text{Risk} = \text{High}) \cdot P(\text{Risk} = \text{High}) = \frac{1}{3} \cdot \frac{1}{3} \cdot 0.75 \cdot 0.5 \cdot \frac{7}{17} \approx 0.017$$

#### Low Risk:

$$P(\text{Risk} = \text{Low}) = \frac{9 + 1}{15 + 2} = \frac{10}{17} \approx 0.588$$

$$P(\text{Income} = \text{Medium} \mid \text{Risk} = \text{Low}) = \frac{3 + 1}{9 + 3} = \frac{1}{3} \approx 0.333$$

$$P(\text{Occupation} = \text{Transport} \mid \text{Risk} = \text{Low}) = \frac{2 + 1}{9 + 3} = \frac{1}{4} = 0.25$$

$$P(\text{Marital Status} = \text{Single} \mid \text{Risk} = \text{Low}) = \frac{3 + 1}{9 + 2} = \frac{4}{11} \approx 0.364$$

$$P(\text{House Ownership} = \text{Yes} \mid \text{Risk} = \text{Low}) = \frac{6 + 1}{9 + 2} = \frac{7}{11} \approx 0.636$$

$$P(X \mid \text{Risk} = \text{Low}) \cdot P(\text{Risk} = \text{Low}) = \frac{1}{3} \cdot 0.25 \cdot \frac{4}{11} \cdot \frac{7}{11} \cdot \frac{10}{17} \approx 0.011$$

**Conclusion:** Profile 1 is classified as **High Risk**.

#### Profile 2:

$X$

= {Income = High, Occupation = Media, Marital Status = Single, House Ownership = Yes}

#### High Risk:

$$P(\text{Income} = \text{High} \mid \text{Risk} = \text{High}) = \frac{1 + 1}{6 + 3} = \frac{2}{9} \approx 0.222$$

$$P(\text{Occupation} = \text{Media} \mid \text{Risk} = \text{High}) = \frac{2 + 1}{6 + 3} = \frac{1}{3} \approx 0.333$$

$$P(\text{Marital Status} = \text{Single} \mid \text{Risk} = \text{High}) = \frac{5 + 1}{6 + 2} = \frac{3}{4} = 0.75$$

$$P(\text{House Ownership} = \text{Yes} \mid \text{Risk} = \text{High}) = \frac{3 + 1}{6 + 2} = \frac{1}{2} = 0.5$$

$$P(X \mid \text{Risk} = \text{High}) \cdot P(\text{Risk} = \text{High}) = \frac{2}{9} \cdot \frac{1}{3} \cdot 0.75 \cdot 0.5 \cdot \frac{7}{17} \approx 0.011$$

**Low Risk:**

$$P(\text{Income} = \text{High} \mid \text{Risk} = \text{Low}) = \frac{4 + 1}{9 + 3} = \frac{5}{12} \approx 0.417$$

$$P(\text{Occupation} = \text{Media} \mid \text{Risk} = \text{Low}) = \frac{4 + 1}{9 + 3} = \frac{5}{12} \approx 0.417$$

$$P(\text{Marital Status} = \text{Single} \mid \text{Risk} = \text{Low}) = \frac{3 + 1}{9 + 2} = \frac{4}{11} \approx 0.364$$

$$P(\text{House Ownership} = \text{Yes} \mid \text{Risk} = \text{Low}) = \frac{6 + 1}{9 + 2} = \frac{7}{11} \approx 0.636$$

$$P(X \mid \text{Risk} = \text{Low}) \cdot P(\text{Risk} = \text{Low}) = \frac{5}{12} \cdot \frac{5}{12} \cdot \frac{4}{11} \cdot \frac{7}{11} \cdot \frac{10}{17} \approx 0.024$$

**Conclusion:** Profile 2 is classified as **Low Risk**.

**Profile 3:**

$X$

= {Income = Low, Occupation = Transport, Marital Status = Married, House Ownership = Yes}

**High Risk:**

$$P(\text{Income} = \text{Low} \mid \text{Risk} = \text{High}) = \frac{3 + 1}{6 + 3} = \frac{4}{9} \approx 0.444$$

$$P(\text{Occupation} = \text{Transport} \mid \text{Risk} = \text{High}) = \frac{2 + 1}{6 + 3} = \frac{1}{3} \approx 0.333$$

$$P(\text{Marital Status} = \text{Married} \mid \text{Risk} = \text{High}) = \frac{1 + 1}{6 + 2} = \frac{1}{4} = 0.25$$

$$P(\text{House Ownership} = \text{Yes} \mid \text{Risk} = \text{High}) = \frac{3 + 1}{6 + 2} = \frac{1}{2} = 0.5$$

$$P(X \mid \text{Risk} = \text{High}) \cdot P(\text{Risk} = \text{High}) = \frac{4}{9} \cdot \frac{1}{3} \cdot 0.25 \cdot 0.5 \cdot \frac{7}{17} \approx 0.007$$

**Low Risk:**

$$P(\text{Income} = \text{Low} \mid \text{Risk} = \text{Low}) = \frac{2 + 1}{9 + 3} = \frac{1}{4} = 0.25$$

$$P(\text{Occupation} = \text{Transport} \mid \text{Risk} = \text{Low}) = \frac{2 + 1}{9 + 3} = \frac{1}{4} = 0.25$$

$$P(\text{Marital Status} = \text{Married} \mid \text{Risk} = \text{Low}) = \frac{6 + 1}{9 + 2} = \frac{7}{11} \approx 0.636$$

$$P(\text{House Ownership} = \text{Yes} \mid \text{Risk} = \text{Low}) = \frac{6 + 1}{9 + 2} = \frac{7}{11} \approx 0.636$$

$$P(X \mid \text{Risk} = \text{Low}) \cdot P(\text{Risk} = \text{Low}) = \frac{1}{4} \cdot \frac{1}{4} \cdot \frac{7}{11} \cdot \frac{7}{11} \cdot \frac{10}{17} \approx 0.014$$

**Conclusion:** Profile 3 is classified as **Low Risk**.

## VI. Exercises

**Exercise 1.** A toy manufacturing company wants to predict the **sales** of upcoming products. The collected dataset is as follows:

| Type    | Number of Colors | Size   | Material   | Sales       |
|---------|------------------|--------|------------|-------------|
| Control | 3                | Small  | PP Plastic | <i>High</i> |
| Puzzle  | 5                | Medium | Rubber     | <i>Low</i>  |
| Puzzle  | 7                | Large  | PP Plastic | <i>Low</i>  |
| Control | 5                | Small  | Rubber     | <i>Low</i>  |
| Doll    | 3                | Medium | PP Plastic | <i>Low</i>  |
| Control | 5                | Medium | PP Plastic | <i>High</i> |
| Doll    | 5                | Large  | PP Plastic | <i>High</i> |
| Control | 7                | Medium | Rubber     | <i>Low</i>  |
| Puzzle  | 7                | Large  | Rubber     | <i>High</i> |
| Puzzle  | 3                | Large  | PP Plastic | <i>Low</i>  |
| Doll    | 3                | Small  | Rubber     | <i>Low</i>  |
| Puzzle  | 3                | Small  | PP Plastic | <i>High</i> |
| Control | 5                | Large  | Rubber     | <i>Low</i>  |
| Doll    | 5                | Medium | PP Plastic | <i>High</i> |
| Doll    | 7                | Large  | PP Plastic | <i>High</i> |

**Question 1:** Compute Information Gain of attributes and build a decision tree using ID3.

**Hint:**

# Compute entropy

```

def entropy(labels):
    from collections import Counter
    counter = Counter(labels)
    total = len(labels)
    return -sum((c/total)*math.log2(c/total) for c in counter.values())

# Compute information gain
def info_gain(data, attr):
    total_entropy = entropy([d["Label"] for d in data])
    values = set(d[attr] for d in data)
    weighted_entropy = 0
    for v in values:
        subset = [d for d in data if d[attr] == v]
        weighted_entropy += (len(subset)/len(data)) * entropy([d["Label"] for d in subset])

    return total_entropy - weighted_entropy

# Compute IG for each attribute
for attr in ["Type", "Colors", "Size", "Material"]:
    print(attr, info_gain(data, attr))

```

**Question 2: Compute Gini index and build a decision tree using CART.**

**Hint:**

```

def gini(labels):
    from collections import Counter
    counter = Counter(labels)
    total = len(labels)
    return 1 - sum((c/total)**2 for c in counter.values())

def gini_split(data, attr):
    values = set(d[attr] for d in data)
    g = 0
    for v in values:
        subset = [d for d in data if d[attr] == v]
        g += (len(subset)/len(data)) * gini([d["Label"] for d in subset])
    return g

```

```
for attr in ["Type", "Colors", "Size", "Material"]:
    print(attr, gini_split(data, attr))
```

**Question 3:** Use one of the decision trees to predict sales:

| Type    | Colors | Colors | Material   |
|---------|--------|--------|------------|
| Doll    | 3      | Large  | Rubber     |
| Puzzle  | 5      | Large  | PP Plastic |
| Control | 3      | Large  | Rubber     |

**Hint:**

```
# After building tree → write predict function
def predict(tree, sample):
    if type(tree) != dict:
        return tree
    attr = list(tree.keys())[0]
    value = sample[attr]
    if value in tree[attr]:
        return predict(tree[attr][value], sample)
    else:
        return "Unknown"

# Example
sample = {"Type": "Doll", "Colors": 3, "Size": "Large", "Material": "Rubber"}
print(predict(tree, sample))
```

**Question 4:** Given actual sales: **Low, Low, High**, build confusion matrix and compute Accuracy and Recall

**Hint:**

```
from sklearn.metrics import confusion_matrix, accuracy_score, recall_score

y_true = ["Low", "Low", "High"]
y_pred = ["...", "...", "..."] # results from tree
cm = confusion_matrix(y_true, y_pred, labels=[" High ", " Low "])
print(cm)
acc = accuracy_score(y_true, y_pred)
recall = recall_score(y_true, y_pred, pos_label=" High ")
```

```
print("Accuracy:", acc)
print("Recall:", recall)
```

**Question 5:** Compute prior probability of "Puzzle":

**Hint:**

```
count = sum(1 for d in data if d["Type"] == "Puzzle")
prob = count / len(data)
print(prob)
```

**Question 6:** Compute:  $P(\text{Material} = \text{Rubber} \mid \text{Sales} = \text{Low})$

**Hint:**

```
# P(Material = Rubber | Sales = Low)

subset = [d for d in data if d["Label"] == "Low "]
count = sum(1 for d in subset if d["Material "] == "Rubber ")
prob = count / len(subset)
print(prob)
```

**Question 7:** Use Naïve Bayes + Laplace smoothing for question 3

**Hint:**

```
from collections import Counter

# Count labels
label_counts = Counter(d["Label"] for d in data)

# Laplace smoothing
def prob(attr, value, label):
    subset = [d for d in data if d["Label"] == label]
    count = sum(1 for d in subset if d[attr] == value)

    # Laplace
    k = len(set(d[attr] for d in data))
    return (count + 1) / (len(subset) + k)
```

```

# Compute probability for each class
def predict_nb(sample):
    probs = {}
    for label in ["High", "Low"]:
        p = label_counts[label] / len(data)
        for attr in sample:
            p *= prob(attr, sample[attr], label)
        probs[label] = p
    return max(probs, key=probs.get)

```

**Question 8:** Evaluate Naive Bayes:

**Hint:**

```

y_pred_nb = [predict_nb(s) for s in test_samples]
cm = confusion_matrix(y_true, y_pred_nb, labels=["High", "Low"])
acc = accuracy_score(y_true, y_pred_nb)
recall = recall_score(y_true, y_pred_nb, pos_label="High")

```

**Question 9:** Compare Decision Tree vs Naive Bayes:

**Hint:**

```

print("Decision Tree Accuracy:", acc_tree)
print("Naive Bayes Accuracy:", acc_nb)

```

**Question 10:** New product:

| Type   | Colors | Size  | Material |
|--------|--------|-------|----------|
| Puzzle | 7      | Small | Rubber   |

Use the trained models to predict the company's **sales** for this product.

**Hint:**

```

new_sample = {"Type ": "Puzzle ", "Colors ": 7, "Size ": "Small ", "Material ": "Rubber "}
print("Tree:", predict(tree, new_sample))
print("Naive Bayes:", predict_nb(new_sample))

```

**Exercise 2.** Red Wine Quality Dataset quality

Convert to binary classification:

- $\text{quality} \geq 6 \rightarrow \text{Good (1)}$
- $\text{quality} < 6 \rightarrow \text{Bad (0)}$

**Question 1:** Data preprocessing

1. Read CSV
2. Check missing & duplicates
3. Convert label
4. Split X, y
5. Train/Test = 80/20

**Question 2:** Build a Decision Tree model using the Gini index

**Steps:**

1. Train the model
2. Make predictions on the test set

Evaluate using the following metrics: Accuracy, Precision, Recall, F1-score

Hint:

```
models["Decision Tree"] = Pipeline([
    ("clf", DecisionTreeClassifier(
        criterion="gini",
        max_depth=4,
        random_state=RANDOM_STATE
    ))
])
```

**Question 3:** Use Gaussian Naive Bayes

**Steps:**

1. Train the model
2. Make predictions on the test set

Evaluate using the following metrics: Accuracy, Precision, Recall, F1-score

Hint:

```
models["Naive Bayes"] = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", GaussianNB())
])
```

### Questions:

- Which model performs better?
- Why is there a difference?

### Question 4: Build a Random Forest model

#### Steps:

Train the model with different parameters: `n_estimators` và `max_depth`

Evaluate using: Accuracy, Precision, Recall, F1-score.

Hint:

```
models["Random Forest"] = Pipeline([
    ("clf", RandomForestClassifier(
        n_estimators=200,
        max_depth=8,
        random_state=RANDOM_STATE
    ))
])
```

### Question 5: Build an XGBoost model using K-Fold Cross Validation (k = 5 or 10)

Train with parameters: `learning_rate`, `max_depth`, `n_estimators`

Hint:

```
models["XGBoost"] = Pipeline([
    ("clf", XGBClassifier(
```

```

n_estimators=200,
max_depth=4,
learning_rate=0.05,
subsample=0.9,
colsample_bytree=0.9,
objective="binary:logistic",
eval_metric="logloss",
random_state=RANDOM_STATE
))
])

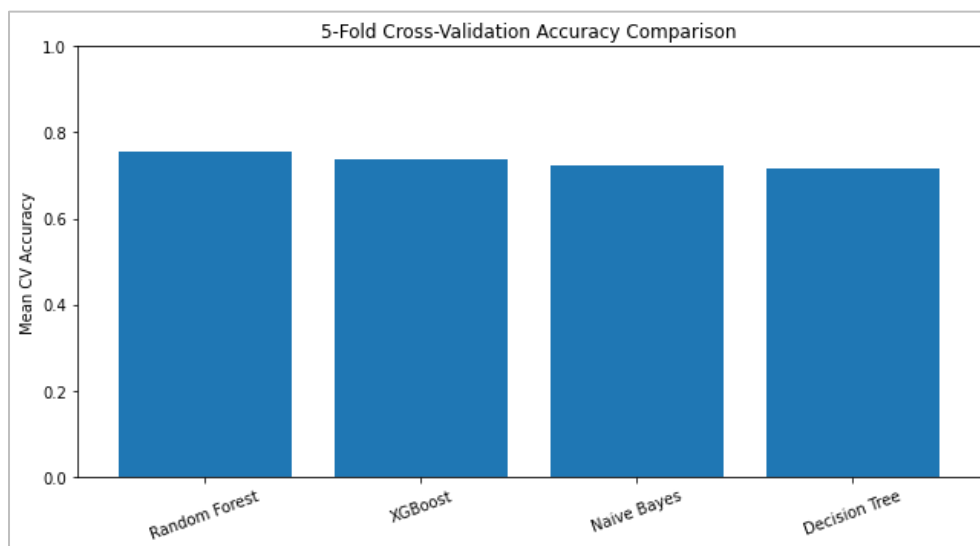
```

Evaluate using: Accuracy, Precision, Recall, F1-score

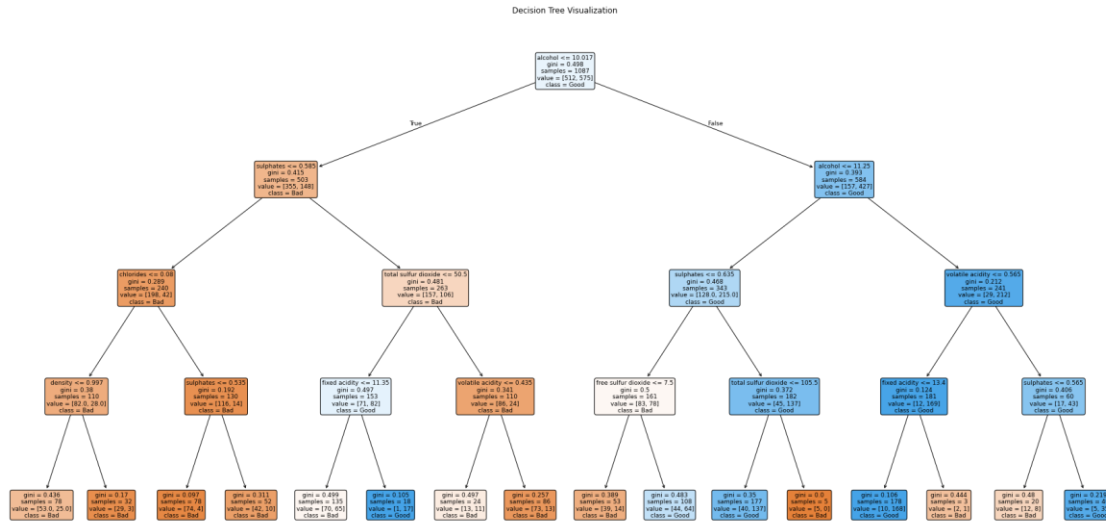
Then apply **K-Fold Cross Validation** for the three models above:

- Compute mean Accuracy
- Compute standard deviation

Finally, visualize the decision tree.



## Decision Tree Illustration:



## Question 6: Plot feature importance

Compare between: Decision Tree, Random Forest, XGBoost

Hint:

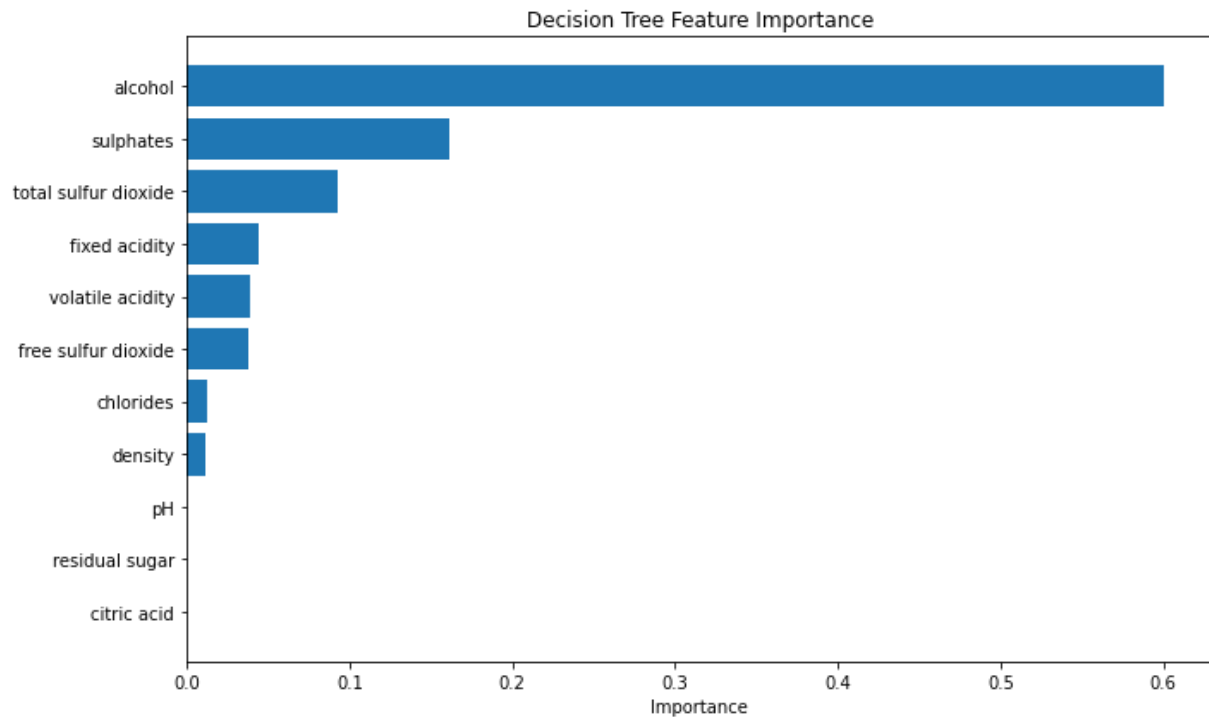
```
importances = model.feature_importances_
```

```
idx = np.argsort(importances)[::-1]
```

```
sorted_features = [feature_names[i] for i in idx]
```

```
sorted_importances = importances[idx]
```

Example Result:



**Exercise 3.** The **Heart Disease dataset** contains 14 attributes collected in Cleveland, USA.

The attribute '**target**' is an integer with values:

- 0 (absence of disease)
- 1 (presence of heart disease in the patient)

**Question 1:** Load the input data and perform preprocessing:

- Remove rows with missing values
- Rename the columns for easier tracking as follows: 'age', 'sex', 'chest\_pain\_type', 'resting\_blood\_pressure', 'cholesterol', 'fasting\_blood\_sugar', 'rest\_ecg', 'max\_heart\_rate\_achieved', 'exercise\_induced\_angina', 'st\_depression', 'st\_slope', 'num\_major\_vessels', 'thalassemia', 'target'

**Question 2, 3, 4, 5, 6:** Same as the questions in **Exercise 2**.